Chapter 5

Exception Handling

Exception Handling

- *An exception is an error that interrupts the execution of a program.*
- *Exception handling in java allows you to catch and handle runtime errors or exceptional conditions that occur during the execution of your program.*
- *It helps you to handle errors and prevent your program from crashing.*
- *Some examples:*
  - *Performing illegal arithmetic operation like division by zero.*
  - *Illegal arguments to methods*
  - *Accessing an out-of-bounds array element*
  - *Writing to a read-only file*
  - *Accessing a missed file*
- *An exception can occur for many different reasons, including the following:*
  - *A user has entered invalid data,*
  - *A file that needs to be opened cannot be found,*
  - *A network connection has been lost in the middle of communications,*
- *All of these problems have to be handled using exception handling.*

Lets have the following small program

```
public class UncougtException {
 public static void main(String args[])
 {
 int x = 0, y = 10;
 int z = y / x;
 System.out.println(z);
 }}
```

- When we try to execute the above program it will interrupt. since, division by zero is illegal mathematical operation.
- ArithmeticException: DivisionByZero exception will raise

```
run:
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Exceptiontry.UncougtException.main(UncougtException.java:11)
Java Result: 1
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
public class UncougtException {
 public static void main(String args[])
 {
   String[] grade = {"A","B","C"};
   System.out.println(grade[3]);
 }
}
```

- When you try to execute again it will be interrupted.
- System.out.println try to access an array element that doesn't exist. And an ArrayIndexOutOfBoundException will arise.

```
run:
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 3
        at Exceptiontry.UncougtException.main(UncougtException.java:11)
Java Result: 1
BUILD SUCCESSFUL (total time: 0 seconds)
```

Exception Handling keyword

- *In Java, there are several keywords to handle exceptions. The most common are:*

1. ***try:*** *The try block is used to enclose the code/statements that might throw an exception. It is followed by one or more catch blocks or a finally block. Example:*

```
try {
  // Code that might throw an exception
 int x = 0, y = 10;
 int z = y / x;
System.out.println("z: "+ result);
}
catch (ArithmeticException e) {
   // Code to handle the exception
   System.out.println("Error: " + e.getMessage());
}
```

//output Error: / by zero

```
try
   {
   String[] grade = {"A","B","C"};
   System.out.println(grade[3]);
    }
    catch(Exception e)
    {
System.out.println("Error:"+e.getM
essage());
    }
```

//output: Error:3

Exception Handling keyword

2. Catch: *The catch keyword is used to catch and handle exceptions. It follows the try block and specifies the type of exception it can handle. Multiple catch blocks can be used to handle different types of exceptions. Here's an example:*

```
try {
   // Code that might throw an exception
   int x=0,y=10;
   int result = y/x;
   System.out.println("Result: " + result);
} catch (ArithmeticException e) {
   // Code to handle ArithmeticException
   System.out.println("ArithmeticException: " + e.getMessage());
} catch (ArrayIndexOutOfBoundsException e) {
   // Code to handle ArrayIndexOutOfBoundsException exceptions
   System.out.println("Exception: " + e.getMessage());
}
```

```
ArithmeticException: / by zero
```

Exception Handling keyword

3. finally: *the finally keyword is used to define a block of code that is always executed, regardless of whether an exception occurred or not. Here's an example:*

```
try {
   // Code that might throw an exception
   int x=0,y=10;
   int result = y/x;
   System.out.println("Result: " + result);
}catch (ArithmeticException e) {
   // Code to handle the exception
   System.out.println("Error: " + e.getMessage());
}finally {
   // Code that always executes
   System.out.println("Finally block executed");
}}
//output:
Error: / by zero
Finally block executed
```

Exception Handling keyword

4. throw *– The throw keyword is used to explicitly throw a specific exception from the program.*

```
public class ThrowExample {
   public static void main(String[] args) {
      try {
         int result = divide(10, 0);
         System.out.println("Result: " + result);
      } catch (ArithmeticException e) {
         System.out.println("Error: " + e.getMessage());
      }
   }
   public static int divide(int dividend, int divisor) {
      if (divisor == 0) {
         throw new ArithmeticException("Division by zero");
      }
      return dividend / divisor;
   }}
```

Exception Handling keyword

5. throws – *The throws keyword is used in a method declaration to specify that the method might throw certain types of exceptions. It allows the calling method to handle the exceptions or propagate them further.* ***Syntax:***

modifiers returnType methodName(parameters) throws ExceptionType1, ExceptionType2, … {

// Method body

}

```
import java.io.FileNotFoundException;
import java.io.FileReader;
public class ThrowsExample {
    public static void main(String[] args) {
        try {
            readFromFile("file.txt");
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        }   }
    public static void readFromFile(String filename) throws
FileNotFoundException {
        FileReader fileReader = new FileReader(filename);
        // Code to read from the file
    }}
```

```
File not found: file.txt (The system cannot find the file specified)
```

Exception types

In Java, exceptions are divided into two main categories: checked exceptions and unchecked exceptions.

1. Checked Exceptions:

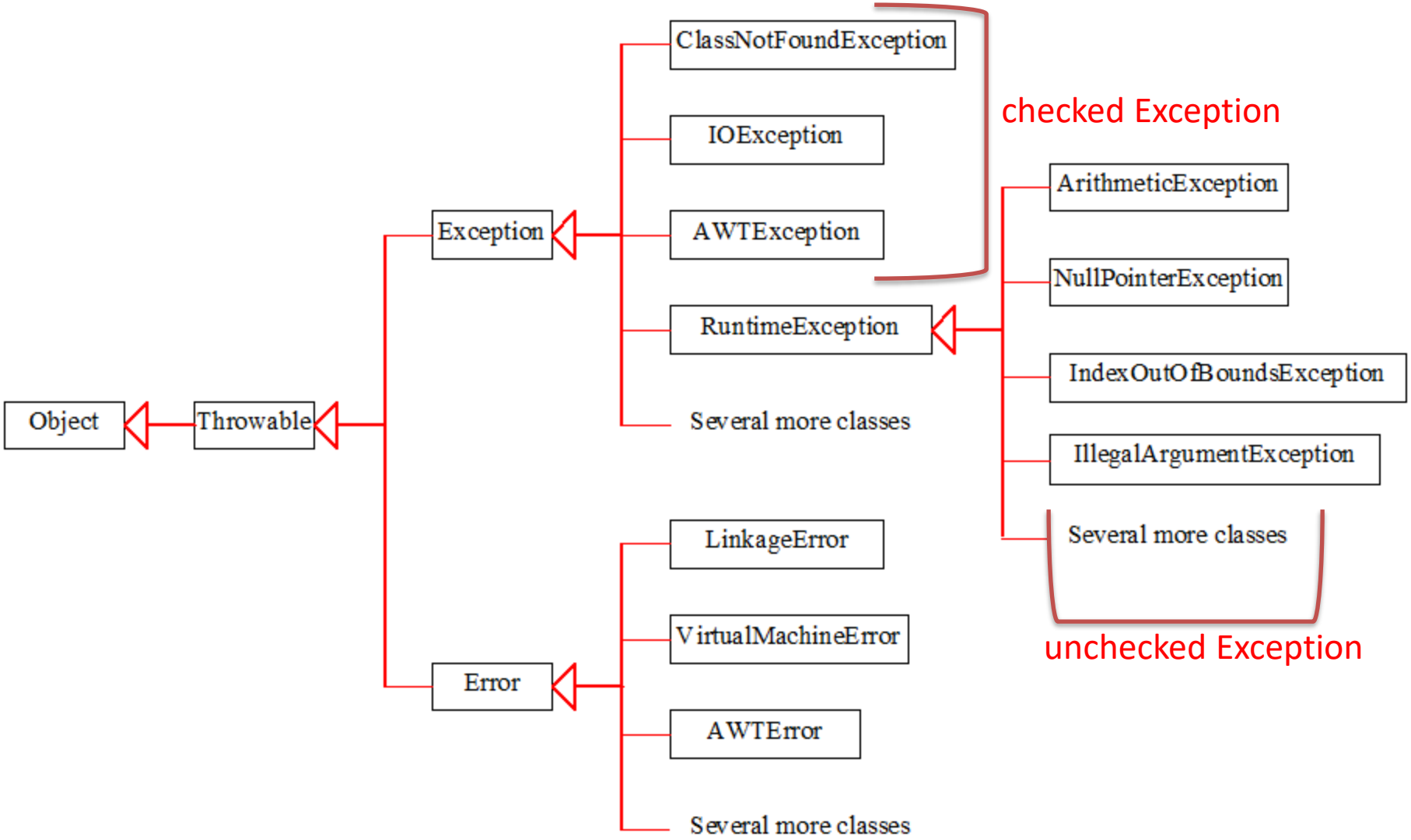
- *Checked exceptions are exceptions that are checked at compile-time. i.e. the compiler enforces that they are either caught or declared to be thrown.*
- *Examples of checked exceptions include **IOException**, **SQLException**, **ClassNotFoundException**, and **FileNotFoundException**.*
  - *These exceptions typically represent error conditions that can occur during input/output operations, database operations, or class loading, and to access a file that does not exist or cannot be opened respectively.*
- *Methods that can throw checked exceptions must declare them using the **throws keyword** in their method signature or in try/catch block.*
- *Checked exceptions are subclasses of the Exception class but not subclasses of RuntimeException.*

Exception types

2. ***Unchecked Exceptions (Runtime Exceptions):***

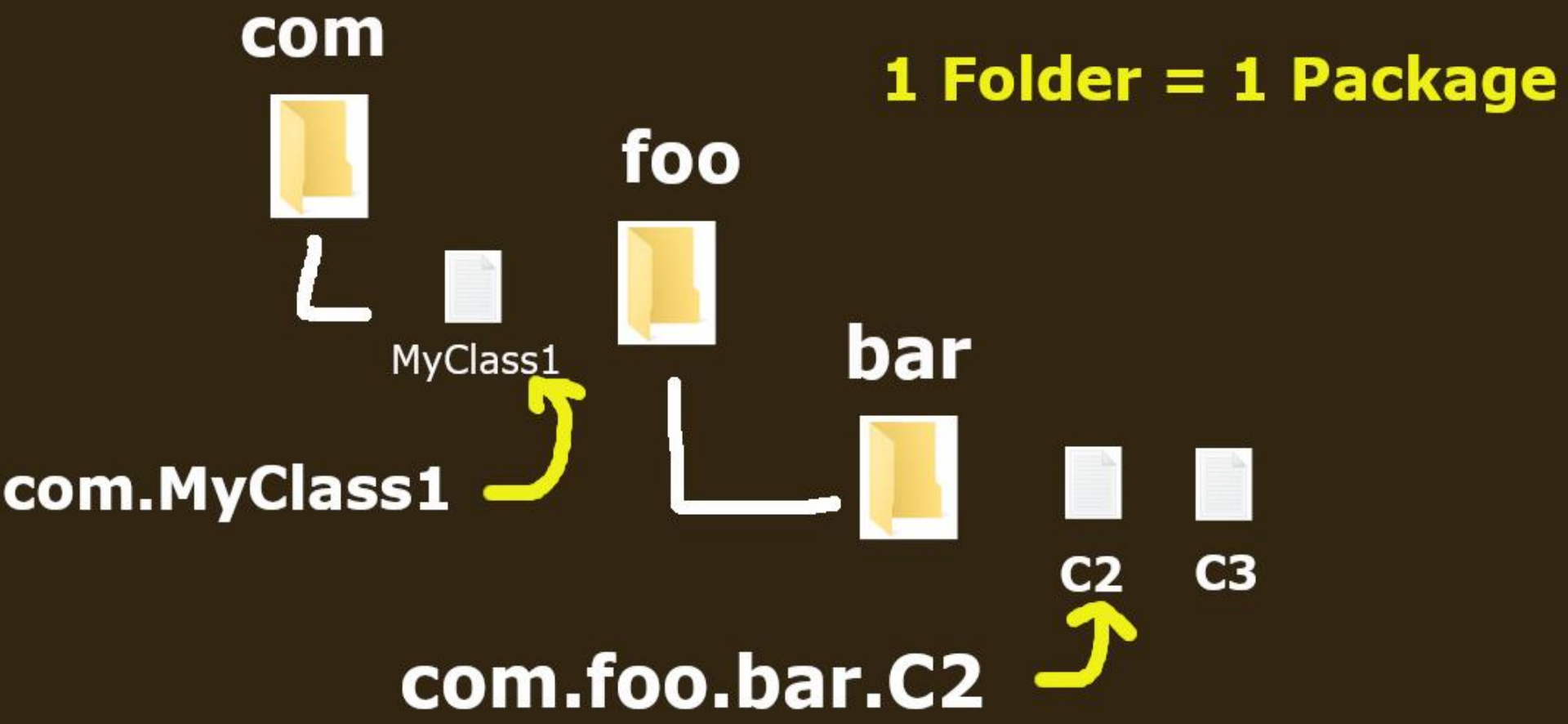
- *Unchecked exceptions are exceptions that are not checked at compile-time.*
- *Unchecked exceptions, also known as* ***runtime exceptions****, are exceptions that do not need to be declared or caught explicitly.*
- *They occur due to programming errors or exceptional conditions that can be detected at runtime.*
- ***Unchecked exceptions*** *extend the RuntimeException class or one of its subclasses.*
- *Example:* ***NullPointerException****,****ArrayIndexOutOfBoundsException****,* ***ArithmeticException****, and* ***IllegalArgumentException.***
  - *These exceptions usually indicate programming errors, such as null references, accessing array elements out of bounds, divisible by zero and passing illegal arguments to methods respectively.*

Exception Classes



Chapter 6

Java Package and Import



Introduction

- In Java, a package is a way to organize and group related classes, interfaces, and other code entities.
- Packages are used in Java, in-order to avoid name conflicts and to control access of class, interface etc.
- A package can be defined as a group of similar types of classes, interface, and sub-package.
- Using package it becomes easier to locate the related classes and it also provides a good structure for projects with hundreds of classes and other files.

Benefits of java package

1. Java package is used to categorize the classes and interfaces to easily maintained.

2. Java package provides access protection.

3. Java package removes naming collision.

4. Packages can be provide re-usability of code.

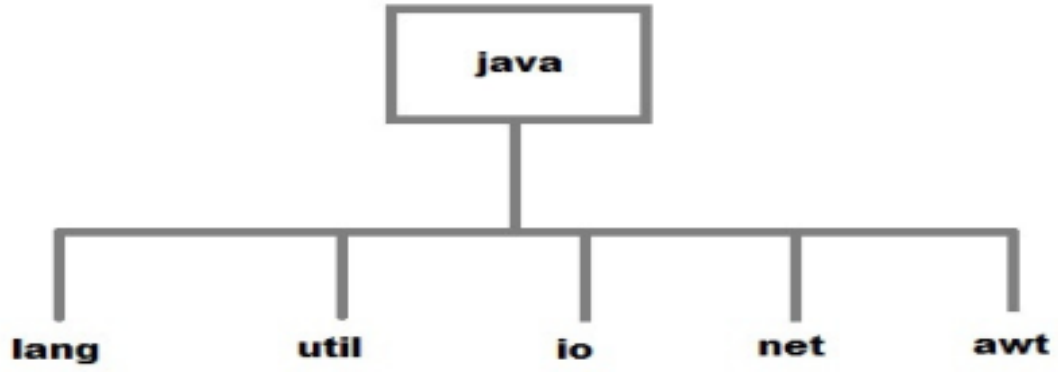
you can create your own package or extend already available package.

Classification

Java packages are classified into two types

1) Java API: existing java package e.g java.util

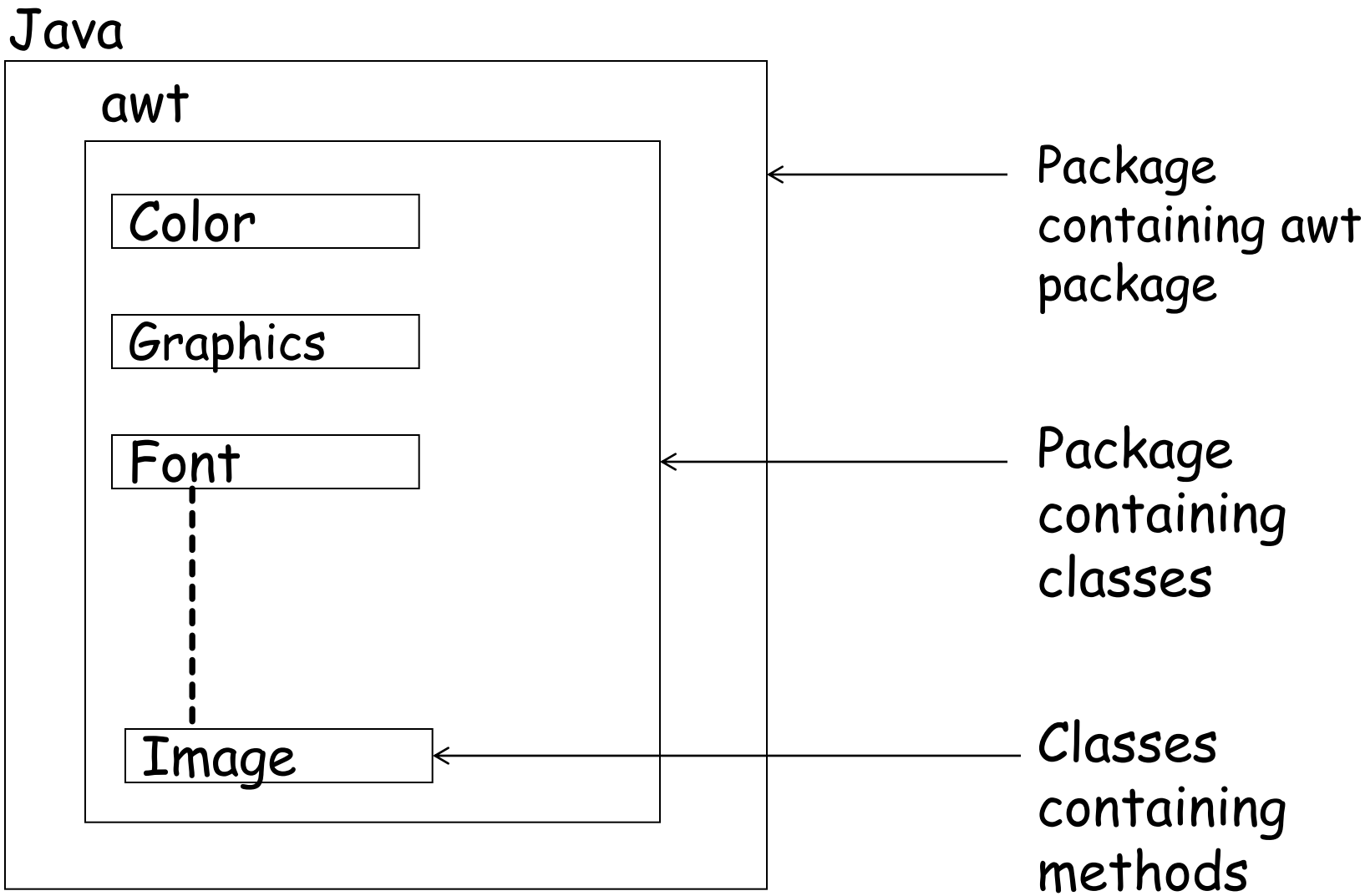
- It refers to a collection of classes, interfaces, and methods provided by the Java Development Kit (JDK) that developers can use to build Java applications.



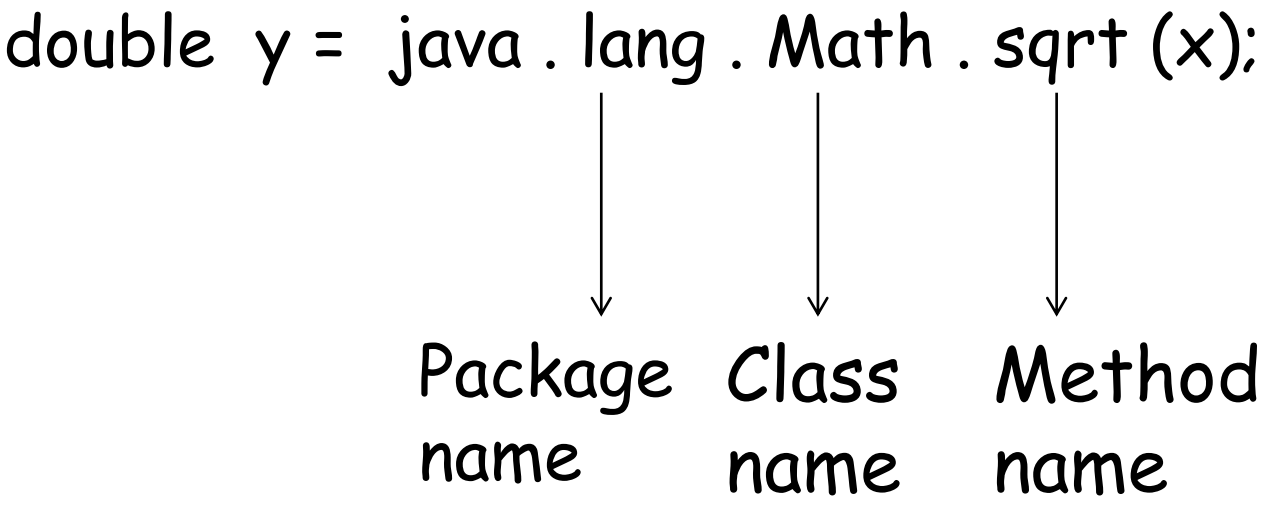
2) User defined packages: java package created by user to categorize their projects classes and interface.

- In Java, user-defined packages are used to organize and group related classes and interfaces together.

Example system package



Naming convention



Creating user-defined package

- To create a user-defined package, you need to follow these steps:
  - Create a new directory with the desired package name.
  - Place the Java source files (classes or interfaces) related to that package in the directory.
  - At the beginning of each Java source file, include the package statement to define the package name.
- For example, let's say you want to create a package called "com.example" for your classes. You would follow these steps:
  - Create a directory structure like com/example/.
  - Place your Java source files (e.g., MyClass.java) inside the com/example/ directory.
  - In each Java source file, include the package statement as the first line: package com.example;.

Creating package example

- Creating a package in java is quite easy. Simply include a package command followed by name of the package as the first statement in java source file.

Example

```
package com.example;
public class MyClass
  {
     ……….
     ……….

     }
```

(Body of class)

Accessing a Package

- There are three ways to access the package from outside the package.
  1. import package.*;
  2. import package.classname;
  3. fully qualified name.

1. Using packagename.*;

- If you use package.* then all the classes and interfaces of this package will be accessible within a specific packages.
- The import keyword is used to make the classes and interface of another package accessible to the other package.

Cont...

- **Example of package that import the packagename.*:**

```
//save by A.java
package pack;
public class A
{
  public void msg()
{
System.out.println("Hello java");}
}
```

```
//save by B.java
package mypack;
import pack.*;
  class B
{
  public static void main(String args[])
{
   A obj = new A();
   obj.msg();
  }
}
```

Output: Hello java

Cont...

2. Using packagename.classname;

- If you import **package.classname** then only declared class of this package will be accessible.

- **Example of package by import package.classname:**

```
//save by A.java

package pack;
public class A
{
  public void msg()
{
System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
import pack.A;

class B
{
  public static void main(String args[])
{
   A obj = new A();
   obj.msg();
  }
}
```

Output: Hello

Cont...

3. Using fully qualified name;

- Instead of importing a package or class, you can use the fully qualified name of a class to refer to it directly. The fully qualified name includes the package name followed by the class name.
- If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

Cont...

- **Example of package by import fully qualified name:**

```
//save by A.java
package pack;
public class A
{
  public void msg()
{
System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
class B
{
  public static void main(String args[])
{
   pack.A obj = new pack.A();   //using fully qualified name
   obj.msg();
  }
}
```

Output: Hello

Thanks

Questions???